CoffeeCare: Crítica pelo SDLC

Projeto de Pesquisa e Inovação, 1º semestre de Ciência da Computação, SPTech.

Veredito geral: a base técnica está boa para o 1º semestre (sensor → API → banco → site). Mas "faltando apenas os slides" é otimista. Pelo que foi descrito, **o requisito 4 (alertas) não aparece como concluído**, não há menção a testes nem a validação com produtor, e o deploy é local.

1. Análise de Requisitos: lacunas médias a altas

- **"Faixa ideal" não está definida.** Qual é a faixa para café arábica? Ela varia com tipo de solo, fase da cultura (floração, granação) e profundidade. Sem fonte (Embrapa, artigos agronômicos), o alerta é arbitrário.
- **Os requisitos não têm critério de aceite.** "Dashboard com dados" é vago. Qual o intervalo de atualização? Mostra histórico de quanto tempo? Em quantos segundos o alerta aparece?
- **Não há requisitos não funcionais:** disponibilidade, segurança, desempenho, uso em campo (celular?).
- **Alerta chega por onde?** Se só aparece no dashboard, o produtor só o vê se estiver com o site aberto. Isso enfraquece o requisito. E-mail ou WhatsApp seriam o mínimo útil.
- **O simulador financeiro é a parte mais frágil em credibilidade.** De onde vem a fórmula? Quais premissas (preço da saca, produtividade, economia de água/insumo)? Ele usa os dados reais do sensor ou é independente? Prometer "ganho" sem premissas explícitas e fonte é o que a banca mais vai questionar.
- **Não há evidência de validação com o público-alvo** (entrevista com produtor, mesmo que uma só). Sem isso, as personas e dores são suposição.

2. Estudo de Viabilidade: o maior ponto fraco

- **Técnica:** Arduino via USB num computador rodando Node funciona como protótipo, mas **não funciona numa lavoura**. Quem tem um PC ligado no meio do cafezal? O caminho real seria ESP32 com Wi-Fi, LoRa ou GSM, alimentado por energia solar. Diga isso como limitação conhecida e roadmap.
- **Um sensor, um ponto:** solo é heterogêneo. Um sensor não representa um talhão. Precisa de múltiplos pontos e profundidades para a decisão ser confiável.
- **Econômica:** não há custo por ponto de medição, retorno estimado nem comparação com alternativas (tensiômetro, estação meteorológica, irrigação por cronograma).

- **Operacional:** conectividade rural, durabilidade do sensor (corrosão, umidade, sol) e quem faz a manutenção.

3. Design

- **Calibração do sensor capacitivo:** ele devolve valores analógicos brutos, não porcentagem. Sem calibração (leitura no ar seco e na água, ou solo seco e saturado), o "% de umidade" do dashboard pode estar errado.
- **Modelagem do banco:** confira se há entidades para **sensor**, **propriedade/talhão**, **faixa ideal** e **alerta (histórico)**. Se só existem usuário e medição, o sistema não escala para mais de um sensor ou propriedade.
- **Segurança:** senha com hash? As queries do cadastro/login usam parâmetros ou concatenação de strings (risco de SQL injection, comum em templates de API)? Credenciais do banco estão em `.env` fora do repositório?
- **Diagrama de solução:** ele reflete a arquitetura real, inclusive o fluxo do alerta?

4. Codificação

- **Frequência de leitura:** umidade do solo muda devagar. Gravar a cada segundo enche a tabela à toa. Algo como 1 a 10 minutos é suficiente (ou gravar só quando houver variação).
- **Tratamento de falhas:** o que acontece se o USB desconecta, o sensor devolve valor absurdo (0 ou 1023) ou o banco cai? A API reconecta ou morre em silêncio?
- **Timestamp e fuso horário** das medições (servidor vs. banco).
- **Validação de entrada** no cadastro (e-mail duplicado, senha fraca, campos vazios), no front e no back.
- **Git:** commits claros, branches por funcionalidade e divisão de tarefas visível no histórico. A banca costuma olhar isso.

5. Teste: praticamente ausente no relato

Faltam, no mínimo:

- **Teste do sensor:** comparar a leitura com um método de referência (por exemplo, pesar amostra de solo antes e depois de secar) ou ao menos testar em solo seco, úmido e encharcado.
- **Teste de longa duração:** deixar rodando 24h ou mais e ver se a coleta é estável.
- **Teste de alerta:** simular valores fora da faixa e medir se e quando o alerta dispara.

- **Teste funcional:** login errado, cadastro duplicado, acesso ao dashboard sem login (a página é protegida de verdade ou basta digitar a URL?).
- **Teste do simulador** com casos de cálculo conhecidos.
- **Teste de usabilidade** com pelo menos uma pessoa fora do grupo.

6. Instalação/Deploy

- Tudo roda **localmente (VM Lubuntu)**: o produtor não acessa o site de lugar nenhum. Defina se a entrega é uma demo local (aceitável, mas diga isso) ou se há hospedagem em nuvem.
- **Ponto único de falha:** VM, API e Arduino dependem da mesma máquina. Considere inicialização automática (pm2/systemd) e rede da VM em modo estável (bridge).
- **Backup do banco** e script de criação/população versionado no GitHub.
- Um **README com passo a passo de instalação** que outra pessoa consiga seguir sozinha.

7. Manutenção

- Plano de recalibração e troca do sensor, atualização de firmware e logs de erro.
- **LGPD:** o sistema guarda dados pessoais (nome, e-mail, possivelmente localização da propriedade). Mencione consentimento e a política de privacidade se o site for público.
- Documentação técnica para quem herdar o projeto.

Top 5 riscos

#	Risco	Impacto
1	Alertas não implementados ou só visuais	Alto (requisito central)
2	Sensor sem calibração e sem teste de precisão	Alto (dado errado = decisão errada)
3	Simulador financeiro sem premissas/fonte	Alto (credibilidade)
4	Arquitetura USB inviável em campo, não assumida como limitação	Médio/alto (pergunta certa da banca)
5	Segurança básica (hash, SQL injection, credenciais)	Médio

Próximos passos antes de fechar os slides

1. **Fechar o requisito 4:** definir a faixa ideal com fonte e implementar o alerta (idealmente também por e-mail).
2. **Calibrar o sensor** e registrar o procedimento e os valores.
3. **Rodar um teste de 24h** e uma bateria curta de testes funcionais, e guardar prints e evidências.
4. **Revisar o simulador:** premissas explícitas, fonte do preço/produtividade e aviso de que é uma estimativa.
5. **Auditar a segurança:** hash de senha, queries parametrizadas, `.env` fora do Git.
6. **Montar nos slides uma seção "Limitações e próximos passos"** (ESP32/LoRa, múltiplos sensores, nuvem, validação com produtor). Isso transforma as fraquezas em maturidade técnica.
7. **Ensaiar a demo** com plano B (vídeo gravado), porque hardware ao vivo falha.